

大模型原理系列

深入理解Transformer架构

AI技术专栏

2026年

目录

1. 大模型原理系列 01
2. 大模型原理系列 03 · 位置编码
3. 大模型原理系列 03
4. 大模型原理系列 04
5. 大模型原理系列 05
6. 大模型原理系列 06
7. 大模型原理系列 07
8. 大模型原理系列 08
9. 大模型原理系列 · 开篇
10. 大模型原理系列 · 总结

1. 大模型原理系列 01

大模型原理系列 01

系列四从RNN到Transformer，模型是怎么"学会"理解语言的

1从系列三的一个问题开始

在系列三的第9篇（章17-18）。

Agent执行引擎里，我们讲了一个请求进来后，Agent是怎么调度的：接收消息、加载记忆、调用工具、生成回复。

但有一个问题我们没有深入讲：模型是怎么"生成"每一个字的？

当你对Agent说"你好"，模型是如何决定下一个字是"啊"还是"呀"？

要回答这个问题，就要从Transformer架构说起。

2RNN时代：串行处理的困境

在Transformer出现之前，主流的序列处理模型是RNN（循环神经网络）。

RNN的工作方式很简单：处理一句话时，它是一个字一个字的处理，就像工厂流水线：

- 读到"你"，记住"你"的信息
- 读到"好"，结合之前的"你"，生成"好"的信息
- 读到"啊"，结合之前的"你"和"好"，生成"啊"的信息



图1：RNN串行处理——必须等待前一个字完成才能处理下一个

问题在于：RNN是串行处理的。"你"和"好"的位置隔得越，模型在生成"啊"的时候就越难记住"你"的信

息。

问题在于：RNN是串行处理的。"你"和"好"的位置隔得越，模型在生成"啊"的时候就越难记住"你"的信息。

这就是为什么早期的聊天机器人经常"前言不搭后语"——模型确实记住了，但记不住太的东西。

还有一个致命问题：无法并行计算。处理一句话需要等前一个字处理完，效率极低。

3Attention机制：让每个字都看到所有字

2017年，Google发表了一篇划时代的论文《Attention Is All You Need》，提出了Transformer架构。

Transformer的核心思想是：不再串行处理，而是并行处理——让每个字同时"看到"整句话。

怎么做到的呢？靠的是Self-Attention（自注意力机制）。

Self-Attention的工作方式是：当你看到一句话时，不是简单地记住每个字，而是计算每个字和其他字之间的关系。

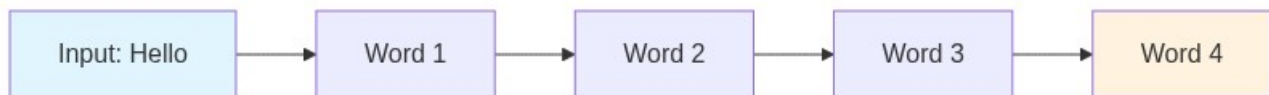


图2：Self-Attention并行处理——所有字同时参与计算

简单理解：Self-Attention就是在问——"这句话里，每个字应该多大程度地参考其他字来理解自己？"

简单理解：Self-Attention就是在问——"这句话里，每个字应该多大程度地参考其他字来理解自己？"

比如在"小明喜欢小红"这句话里：

- "喜欢"这个词，会高度关注"小明"和"小红"（谁喜欢谁）
- "小明"会关注"喜欢"和"小红"（知道自己在喜欢）
- "小红"会关注"小明"和"喜欢"（知道被谁喜欢）

这种"同时看到"的能力，解决了RNN的长期依赖问题。

4Position Encoding：解决"顺序"问题

但Attention有一个问题：它本身不考位置信息。

"小明喜欢小红"和"小红喜欢小明"，在Self-Attention看来是一样的——因为它只看词与词之间的关系，不看谁在谁前面。

Transformer的解决方案是Position

Encoding（位置编码）：给每个位置一个独特的"位置向量"，加到词的表示上。

为什么要用sin/cos函数？因为这样可以让模型很容易学到"相对位置"——比如"在...后面"这样的关系。

为什么要用sin/cos函数？因为这样可以让模型很容易学到"相对位置"——比如"在...后面"这样的关系。

加上位置编码后，"小明喜欢小红"和"小红喜欢小明"就变成了不同的输入。

5Encoder和Decoder：GPT和BERT的区别

完整的Transformer包含两个部分：

- Encoder（编码器）：把输入序列变成一个"理解"——适合完形填空、分类任务
- Decoder（解码器）：基于Encoder的"理解"和已经生成的内容，生成下一个字——适合翻译、对话

BERT是纯Encoder架构——它看完整个句子后，理解了含义，适合做"填空题"（比如判断一句话是正面还是负面）。

GPT是纯Decoder架构——它的任务是逐字生成，每次生成一个字，然后把新生成的字加入输入，再生成下一个字。

这就是为什么GPT是"生成式"的，而BERT不是。

这就是为什么GPT是"生成式"的，而BERT不是。

所以当你和Agent对话时，模型实际上在做这样的循环：

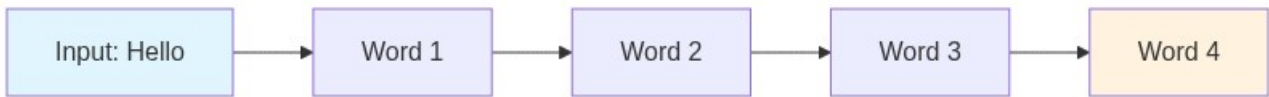


图3：GPT自回归生成——每次生成一个字，加入输入后继续生成

6回到系列三：它和Agent执行引擎的关系

现在我们回头看系列三的内容，能更好地理解Agent是怎么工作的。

执行引擎（17篇）调度模型生成回复，但生成的过程是由Transformer完成的。

记忆系统（8、19篇）把历史对话作为输入传给模型，而Transformer能同时处理整段上下文，这就是Attention的功劳。

工具调用（13篇）时，模型需要理解"现在需要调用一个计算器"，这个理解能力来自Transformer对上下文的分析。

·第9篇（章17-18）· Agent执行引擎—— 调度与执行

·第8篇· 记忆系统概述—— 四层记忆架构

7为什么Transformer这么重要

Transformer的革命性在于三点：

- 并行计算：不再需要等待，可以同时处理所有字，速度提升几十倍
- 长期依赖：无论词相隔多，都能直接建立联系，不再“健忘”
- 可扩展：模型越大，效果越好，这在此前的RNN时代是不可想象的

正是因为Transformer的出现，才有了GPT、BERT、T5等大模型，才有了今天的ChatGPT、Claude、Agent。

8写在最后

Transformer解决了"怎么让模型理解语言"的问题。但还有一个问题没有解决：

模型是怎么知道"小明喜欢小红"是对的，而不是"小红喜欢小明"？

这涉及到训练过程——模型是怎么从海量文本中学习语言规律的。这正是我们下一篇文章要讲的内容。

下期预告：Attention机制详解——Self-Attention是怎么计算"关注度"的，为什么它能让模型理解上下文？

下期预告：Attention机制详解——Self-Attention是怎么计算"关注度"的，为什么它能让模型理解上下文？

大模型原理系列 · 第1篇理解Transformer，窥见大模型的第一道门

2. 大模型原理系列 03 · 位置编码

我的观点：位置编码是Transformer里最被低估的组件。很多人觉得它就是"加个位置信息"，没什么了不起。但真相是：没有位置编码，GPT-4再大也看不懂任何一句话。

先做个小测试

下面两句话有什么区别？

"我爱你""你爱我"

你会说："废话！意思完全相反！"

但如果让一个没有位置编码的AI模型来看，它的答案是——

一模一样。因为Self-Attention只看"有哪些字"，不看"字的顺序"。三个字完全一样，所以AI觉得两句话是一个意思。

上一篇讲了Self-Attention：所有字同时参与计算，互相"查户口"。

但并行计算带来一个致命缺陷：模型不知道字的顺序。

"我爱你"和"你爱我"——同样的三个字，在Self-Attention看来完全一样！因为每个字都和其他字做了点积，顺序根本不存在。

这对NLP是灾难级的。因为语言的核心就是"顺序"。

冷知识

中文里有很多"顺序决定一切"的例子：上海自来水来自海上——正着读反着读都是同一句话，但意思完全不同。如果没有位置编码，AI连这种文字游戏都玩不明白。

2017年的《Attention Is All You Need》论文给出的方案是：在Self-Attention之前，给每个词的位置信息编码到它的向量里。

核心思路很简单：

如果一个词的embedding向量是 x ，位置信息是 p ，那么输入到Self-Attention的向量就是：

这里的 p 就是位置编码（Positional Encoding），每个位置都有自己独特的编码向量。

老师：位置编码是什么？

小明：就是给每个字贴个"座位号"。

老师：那为什么不用1、2、3这样的座位号？

小明：嗯.....

很多人第一反应：给第一个字编号1，第二个字编号2，第三个字编号3.....不就行了吗？

不行。原因有两个：

第一：序号无法比较。位置1和位置3的距离是2，位置2和位置4的距离也是2。但机器看到的只是数字"1"和"3"、"2"和"4"，它们无法从数字本身推断出"距离相等"这个关系。

第二：序号无法泛化。如果训练时最长句子是100个字，那序号就是1-100。推理时来了一个150个字的句子，模型根本没学过101-150怎么处理。

所以位置编码必须满足三个要求：

- 能让模型感知绝对位置（我是第几个）
- 能让模型感知相对位置（我和别人差多）
- 能泛化到没见过的长度

生活中的例子

想象你在电影院找座位。如果座位号只是1、2、3.....你能找到第8排和第10排中间隔了几排。但如果座位号是随机分配的A、B、C、F.....你就无法判断谁离谁更近。位置编码就是让每个位置有一个"智能编号"，既能找到自己，又能算距离。

Transformer论文用的是正弦和余弦函数，生成位置编码。

公式是这样的：

正弦函数 (sin)

一种周期性波动的数学函数，取值范围是 $[-1, 1]$ 。 $\sin(0)=0$, $\sin(\pi/2)=1$, $\sin(\pi)=0$ 。

类比：就像荡秋千，从最低点荡到最高点再荡回来，这个往复运动就是正弦波。

余弦函数 (cos)

和正弦函数类似，也是周期性波动，取值范围 $[-1, 1]$ 。区别是 $\cos(0)=1$ ，起点不同。

类比：两个人一起荡秋千，一个人先出发（sin），另一个人晚四分之一拍出发（cos）。节奏一样，但起点不同。

公式如下：

看起来复，但其实很好理解。

拆解公式

pos：词在句子中的位置，比如0、1、2、3.....

i：维度的索引，比如第0维、第1维、第2维.....

d：向量的总维度，比如512

$10000^{(2i/d)}$ ：这是一个"缩放因子"，不同维度用不同的缩放——前面的维度变化快（高频），后面的维度变化慢（低频）。

类比：位置编码就像"GPS坐标"。经度和纬度组合起来，能精确标识地球上的任何一个位置。正弦和余弦函数就像经度和纬度——不同频率的正弦波组合起来，能唯一标识任何一个位置。而且GPS坐标有个特点：相邻位置的坐标值相差不大，距离位置的坐标相差很大。位置编码也一样——相邻位置的编码向量相近，距离位置的编码向量差异大。

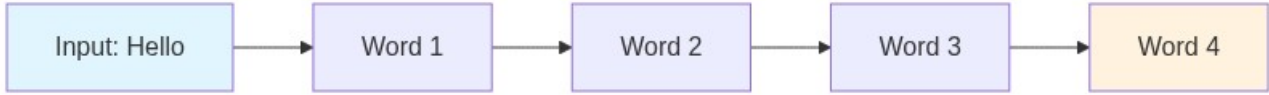
为什么用sin和cos两个函数？

因为sin和cos之间有一个天然关系：

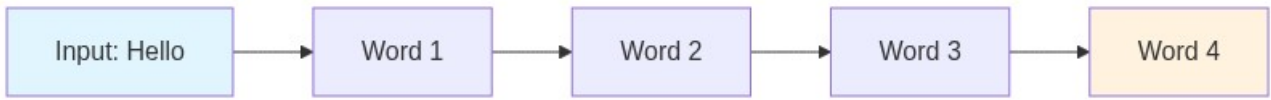
这个性质意味着：位置pos+k的编码，可以通过pos的编码的线性变换得到。换句话说，模型可以很容易地学会"两个位置之间差多少"。

假设我们有一个4维向量（为了简化说明），位置0的位置编码是：

位置1的编码是：



把位置编码加到词向量上，就是最终输入Self-Attention的向量。



老师：你看位置0的编码是[0,1,0,1]，位置1的编码是[0.84,0.54,0.01,1.00]。它们不一样，但也不是完全无关。

小明：就像我和我哥？长得像，但不是同一个人。

老师：这个比喻.....很对。

光看公式不够直观，我们用代码生成位置编码，看看实际效果：

输出：

正弦位置编码有几个关键优势：

特性

说明

为什么重要

唯一性

每个位置都有独特的编码

模型能区分"我爱你"和"你爱我"

相对位置可计算

$PE(pos+k)$ 可由 $PE(pos)$ 线性变换得到

模型能学会"距离"的概念

泛化性

正弦函数是连续的，可以无限延伸

处理比训练时更长的句子也没问题

平滑性

相邻位置的编码向量相似

符合语言的自然规律——近的词关系更近

我的判断：很多人觉得位置编码只是"一个技术细节"，不值得关注。但我的观点恰恰相反——位置编码是Transformer架构中最优雅的设计之一。它用一个简单的数学函数，解决了并行计算的最大痛点。没有它，Self-Attention再强大也无法处理有序文本。

除了正弦位置编码，后来还有很多替代方案：

相对位置编码（Relative Positional Encoding）：直接编码两个词之间的距离，而不是绝对位置。这对某些任务（如机器翻译）效果更好。

旋转位置编码（RoPE）：通过旋转操作编码位置信息，被LLaMA、GPT-J等开源大模型广泛采用。

ALiBi（Attention with Linear Biases）：不用位置编码向量，而是在注意力分数上直接加上位置偏置。这个方案在长文本场景下表现优异。

不同的位置编码方案各有优劣，但正弦位置编码仍然是理解位置编码原理的最佳起点。

彩蛋：RoPE为什么火？

旋转位置编码（RoPE）的核心思想是：把位置信息编码成"旋转角度"。位置0是0度，位置1是旋转30度，位置2是旋转60度……旋转操作有个天然的数学性质：旋转角度相加 = 相对位置可计算。这比原始的正弦编码更直观，所以在开源大模型里特别受欢迎。下次如果你用LLaMA，记得它里面的位置编码就是"转圈圈"。

位置编码的核心思想：在Self-Attention之前，把每个词的位置信息编码成向量，加到词的embedding上。这样模型既知道"这个词是什么"，也知道"这个词在第几个位置"。

下一篇，我们会讲多头注意力（Multi-Head Attention）——让模型从多个角度理解文本，而不是只从一个角度看。

系列三：大模型原理系列01 · Transformer架构原理02 · Self-Attention数学原理03 ·
位置编码：让模型知道"谁在前面" ← 你在这里04 · 多头注意力：让模型从多个角度理解05 ·
Encoder-Decoder架构：大模型的"双脑"结构

3. 大模型原理系列 03

大模型原理系列 03

系列三位置编码 —— 让模型知道"谁在前面"

我的观点：位置编码是Transformer里最被低估的组件。很多人觉得它就是"加个位置信息"，没什么了不起。但真相是：没有位置编码，GPT-4再大也看不懂任何一句话。

我的观点：位置编码是Transformer里最被低估的组件。很多人觉得它就是"加个位置信息"，没什么了不起。但真相是：没有位置编码，GPT-4再大也看不懂任何一句话。

先做个小测试

下面两句话有什么区别？

"我爱你""你爱我"

你会说："废话！意思完全相反！"

但如果让一个没有位置编码的AI模型来看，它的答案是——

一模一样。因为Self-Attention只看"有哪些字"，不看"字的顺序"。三个字完全一样，所以AI觉得两句话是一个意思。

0上期回顾：Self-Attention解决了什么问题

前两期我们讲了Transformer的整体架构，以及Self-Attention机制——让每个词都能"看到"其他所有词，从而理解它们之间的关系。

但还有一个问题没解决：这些词在句子里位置不同，意思就不一样——AI怎么知道谁在前面谁在后面？

1一个致命的问题

上一篇讲了Self-Attention：所有字同时参与计算，互相"查户口"。

但并行计算带来一个致命缺陷：模型不知道字的顺序。

"我爱你"和"你爱我"——同样的三个字，在Self-Attention看来完全一样！因为每个字都和其他字做了点积，顺序根本不存在。

这对NLP是灾难级的。因为语言的核心就是"顺序"。

冷知识

中文里有很多"顺序决定一切"的例子：上海自来水来自海上——正着读反着读都是同一句话，但意思完全不同。如果没有位置编码，AI连这种文字游戏都玩不明白。

2解决方案：位置编码

2017年的《Attention Is All You Need》论文给出的方案是：在Self-Attention之前，给每个词的位置信息编码到它的向量里。

核心思路很简单：

如果一个词的embedding向量是x，位置信息是p，那么输入到Self-Attention的向量就是：

这里的p就是位置编码（Positional Encoding），每个位置都有自己独特的编码向量。

老师：位置编码是什么？

小明：就是给每个字贴个"座位号"。

老师：那为什么不用1、2、3这样的座位号？

小明：嗯.....

3为什么不能用简单的序号？

很多人第一反应：给第一个字编号1，第二个字编号2，第三个字编号3.....不就行了吗？

不行。原因有两个：

第一：序号无法比较。位置1和位置3的距离是2，位置2和位置4的距离也是2。但机器看到的只是数字"1"和"3"、"2"和"4"，它们无法从数字本身推断出"距离相等"这个关系。

第二：序号无法泛化。如果训练时最长句子是100个字，那序号就是1-100。推理时来了一个150个字的句子，模型根本没学过101-150怎么处理。

所以位置编码必须满足三个要求：

- 能让模型感知绝对位置（我是第几个）
- 能让模型感知相对位置（我和别人差多）
- 能泛化到没见过的长度

生活中的例子

想象你在电影院找座位。如果座位号只是1、2、3.....你能找到第8排和第10排中间隔了几排。但如果座位号是随机分配的A、B、C、F.....你就无法判断谁离谁更近。位置编码就是让每个位置有一个"智能编号"，既能找到自己，又能算距离。

4正弦位置编码：最优雅的设计

Transformer论文用的是正弦和余弦函数，生成位置编码。

公式是这样的：

正弦函数（sin）

一种周期性波动的数学函数，取值范围是 $[-1, 1]$ 。 $\sin(0)=0$, $\sin(\pi/2)=1$, $\sin(\pi)=0$ 。

类比：就像荡秋千，从最低点荡到最高点再荡回来，这个往复运动就是正弦波。

余弦函数（cos）

和正弦函数类似，也是周期性波动，取值范围 $[-1, 1]$ 。区别是 $\cos(0)=1$ ，起点不同。

类比：两个人一起荡秋千，一个人先出发（sin），另一个人晚四分之一拍出发（cos）。节奏一样，但起点不同。

公式如下：

看起来复，但其实很好理解。

拆解公式

pos：词在句子中的位置，比如0、1、2、3.....

i：维度的索引，比如第0维、第1维、第2维.....

d：向量的总维度，比如512

$10000^{(2i/d)}$ ：这是一个"缩放因子"，不同维度用不同的缩放——前面的维度变化快（高频），后面的维度变化慢（低频）。

类比：位置编码就像"GPS坐标"。经度和纬度组合起来，能精确标识地球上的任何一个位置。正弦和余弦函数就像经度和纬度——不同频率的正弦波组合起来，能唯一标识任何一个位置。而且GPS坐标有个特点：相邻位置的坐标值相差不大，距离位置的坐标相差很大。位置编码也一样——相邻位置的编码向量相近，距离位置的编码向量差异大。

类比：位置编码就像"GPS坐标"。经度和纬度组合起来，能精确标识地球上的任何一个位置。正弦和余弦函数就像经度和纬度——不同频率的正弦波组合起来，能唯一标识任何一个位置。而且GPS坐标有个特点：相邻位置的坐标值相差不大，距离位置的坐标相差很大。位置编码也一样——相邻位置的编码向量相近，距离位置的编码向量差异大。

为什么用sin和cos两个函数？

因为sin和cos之间有一个天然关系：

这个性质意味着：位置 $\text{pos}+k$ 的编码，可以通过 pos 的编码的线性变换得到。换句话说，模型可以很容易地学会"两个位置之间差多少"。

5位置编码长什么样？

假设我们有一个4维向量（为了简化说明），位置0的位置编码是：

位置1的编码是：

把位置编码加到词向量上，就是最终输入Self-Attention的向量。

老师：你看位置0的编码是 $[0, 1, 0, 1]$ ，位置1的编码是 $[0.84, 0.54, 0.01, 1.00]$ 。它们不一样，但也不是完全无关。

小明：就像我和我哥？长得像，但不是同一个人。

老师：这个比喻……很对。

6用Python代码验证

光看公式不够直观，我们用代码生成位置编码，看看实际效果：

输出：

7为什么这个设计如此巧妙？

正弦位置编码有几个关键优势：

我的判断：很多人觉得位置编码只是"一个技术细节"，不值得关注。但我的观点恰恰相反——位置编码是Transformer架构中最优雅的设计之一。它用一个简单的数学函数，解决了并行计算的最大痛点。没有它，Self-Attention再强大也无法处理有序文本。

我的判断：很多人觉得位置编码只是"一个技术细节"，不值得关注。但我的观点恰恰相反——位置编码是Transformer架构中最优雅的设计之一。它用一个简单的数学函数，解决了并行计算的最大痛点。没有它，Self-Attention再强大也无法处理有序文本。

8其他位置编码方案

除了正弦位置编码，后来还有很多替代方案：

相对位置编码（Relative Positional Encoding）：直接编码两个词之间的距离，而不是绝对位置。这对某些任务（如机器翻译）效果更好。

旋转位置编码（RoPE）：通过旋转操作编码位置信息，被LLaMA、GPT-J等开源大模型广泛采用。

ALiBi（Attention with Linear Biases）：不用位置编码向量，而是在注意力分数上直接加上位置偏置。这个方案在长文本场景下表现优异。

不同的位置编码方案各有优劣，但正弦位置编码仍然是理解位置编码原理的最佳起点。

彩蛋：RoPE为什么火？

旋转位置编码（RoPE）的核心思想是：把位置信息编码成"旋转角度"。位置0是0度，位置1是旋转30度，位置2是旋转60度.....旋转操作有个天然的数学性质：旋转角度相加 = 相对位置可计算。这比原始的正弦编码更直观，所以在开源大模型里特别受欢迎。下次如果你用LLaMA，记得它里面的位置编码就是"转圈圈"。

9总结

位置编码的核心思想：在Self-Attention之前，把每个词的位置信息编码成向量，加到词的embedding上。这样模型既知道"这个词是什么"，也知道"这个词在第几个位置"。

位置编码的核心思想：在Self-Attention之前，把每个词的位置信息编码成向量，加到词的embedding上。这

样模型既知道"这个词是什么"，也知道"这个词在第几个位置"。

下一篇，我们会讲多头注意力（Multi-Head Attention）——让模型从多个角度理解文本，而不是只从一个角度看。

下篇预告

下期我们会讲：多头注意力——为什么AI需要"多个脑子"同时工作？一个角度看问题有什么局限？多头注意力是怎么解决这个问题的？

系列三：大模型原理系列01 · Transformer架构原理02 · Self-Attention数学原理03 ·
位置编码：让模型知道"谁在前面" ← 你在这里04 · 多头注意力：让模型从多个角度理解05 ·
Encoder-Decoder架构：大模型的"双脑"结构

4. 大模型原理系列 04

大模型原理系列 04

系列四多头注意力 —— 让模型从多个角度理解

我的观点：多头注意力是Transformer的灵魂。位置编码让模型知道"谁在前面"，Self-Attention让模型知道"谁和谁有关"，而多头注意力让模型知道——"同一个问题，其实有多个不同的理解方式"。

我的观点：多头注意力是Transformer的灵魂。位置编码让模型知道"谁在前面"，Self-Attention让模型知道"谁和谁有关"，而多头注意力让模型知道——"同一个问题，其实有多个不同的理解方式"。

先问大家一个问题

"苹果"是什么意思？

水果？手机？电脑？

你肯定能答对，因为你会看上下文。

但如果AI只有一个"脑子"来理解，它可能会——

蒙圈了。因为它只有一个角度理解"苹果"，分不清到底是水果还是公司。

0上期回顾：一个关键问题

上期我们讲了位置编码：通过把位置信息加到词向量上，让AI知道"谁在前面"。

但还有一个问题没解决：AI只有一个角度看问题。

回顾一下Self-Attention的公式：

这里的Q、K、V都是通过同一个输入变换而来的。这意味着：一个句子只能从一种"视角"去分析。

生活中的例子

就像你看一幅名画，只用一只眼睛看——看到的是扁平的。但如果你用两只眼睛看，大脑会把两个视角合在一起，变成立体画面。多头注意力就是给AI装上"多只眼睛"。

1解决方案：多头注意力

Multi-Head Attention（多头注意力）的思路很简单：

不要只有一组Q、K、V，而是多组同时算。

核心公式：

翻译成成人话就是：

- 把Q、K、V分别投影到h个不同的空间（h是"头"的数量）
- 每个头独立计算一次Attention
- 把h个结果拼接起来，再做一个线性变换

老师：多头注意力是什么意思？

小明：就是让AI有"多只眼睛"看问题？

老师：很好理解，但不够准确。多头注意力不是"多只眼睛"，而是"多个脑子"。每个头有自己独立的Q、K、V变换，关注不同的信息。

小明：所以每个头可以学到不同的"关注点"？

老师：没错。这就是它的强大之处。

2每个头到底在学什么？

不同的头确实会关注不同类型的信息。研究发现：

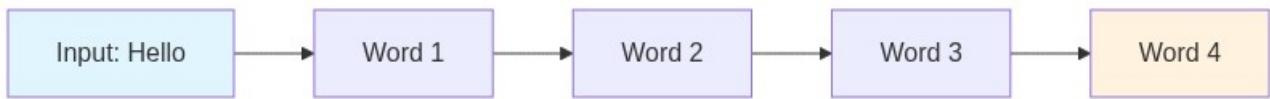


图1：不同注意力头关注不同的信息——有的看语法，有的看指代，有的看语义

3具体例子：一句话的8种理解

比如这句话：

"小明在公园遇到了小红，她正在喂鸽子"

8个"头"会同时分析：

- 头1：谁和谁在互动？（小明小红）
- 头2：她指的是谁？（是小红）
- 头3：在哪里发生的？（公园）
- 头4：在做什么？（喂鸽子）
- 头5：动作的承受者是谁？（鸽子）
- 头6：词和词之间的依赖关系
- 头7：谁在什么地方做什么
- 头8：整句话在讲什么故事

最后，8个头的分析结果合在一起，AI就完全理解了这句话。

4为什么多头比单头更好？

想象你要了解一个人：

生活中的例子

如果只问一个人："这人怎么样？"你只能得到一个角度的评价。但如果同时问他的妈妈、老师、朋友、同事——

- 妈妈说："很孝顺"
- 老师说："很聪明"
- 朋友说："很幽默"
- 同事说："很勤奋"

合起来，你才全面了解这个人。多头注意力就是这个道理。

三个关键优势：

5用代码验证

PyTorch已经内置了Multi-Head Attention，一行代码搞定：

6不同模型的"头"数量

头数不是越多越好，要权衡效果和效率：

我的判断：BERT用12个头，GPT-3用96个头。这不是随便定的——头数越多，每个头能关注的信息越细致，但计算量也越大。BERT是encoder模型，需要理解完整的句子结构；而GPT是decoder模型，需要生成连贯的文本。任务不同，最优的头数也不同。

我的判断：BERT用12个头，GPT-3用96个头。这不是随便定的——头数越多，每个头能关注的信息越细致，但计算量也越大。BERT是encoder模型，需要理解完整的句子结构；而GPT是decoder模型，需要生成连贯的文本。任务不同，最优的头数也不同。

7总结

多头注意力的核心思想：不要只有一组Q、K、V，而是多组同时计算。每个头有自己独立的投影矩阵，关注不同类型的信息。最后把所有头的输出拼接起来，让模型能从多个角度理解文本。

多头注意力的核心思想：不要只有一组Q、K、V，而是多组同时计算。每个头有自己独立的投影矩阵，关注不同类型的信息。最后把所有头的输出拼接起来，让模型能从多个角度理解文本。

一句话记住：多头注意力就是让AI有"多个脑子同时分析"，每个脑子抓一个重点，大家一起把句子读懂。

下一篇，我们会讲Encoder-Decoder架构——大模型的"双脑"结构。

下篇预告

下期我们会讲：Encoder-Decoder架构——大模型是怎么组织的？Encoder（编码器）和Decoder（解码器）分别负责什么？BERT和GPT的区别到底是什么？

系列四：大模型原理系列01 · Transformer架构原理02 · Self-Attention数学原理03 ·
位置编码：让模型知道"谁在前面"04 · 多头注意力：让模型从多个角度理解 ← 你在这里05 ·
Encoder-Decoder架构：大模型的"双脑"结构

5. 大模型原理系列 05

大模型原理系列 05

系列五Encoder-Decoder架构 —— 大模型的"双脑"结构

我的观点：BERT和GPT的区别，本质上是"大脑"的区别。BERT只用了一半——Encoder，用来理解文字；GPT用了另一半——Decoder，用来生成文字。理解了这个问题，才能理解为什么ChatGPT是"生成式AI"，而BERT更适合做"理解型任务"。

我的观点：BERT和GPT的区别，本质上是"大脑"的区别。BERT只用了一半——Encoder，用来理解文字；GPT用了另一半——Decoder，用来生成文字。理解了这个问题，才能理解为什么ChatGPT是"生成式AI"，而BERT更适合做"理解型任务"。

先问大家一个问题

下面两个AI，分别适合做什么？

BERT：输入一句话，输出这句话的情感（正面/负面） GPT：输入一句话，续写后面的内容

你可能会说："这不是一回事吗？"但实际上——

它们用的是完全不同的"大脑"结构！BERT只用了Encoder，GPT只用了Decoder。这节课，我们来搞清楚它们的区别。

0 上期回顾：多头注意力解决了什么问题

上期我们讲了多头注意力：让AI有"多个脑子同时分析"，每个脑子抓一个重点，大家一起把句子读懂。

但还有一个问题没解决：这些"脑子"组织在一起，怎么工作？

比如机器翻译任务：输入一句中文，输出一句英文——这个过程是怎么实现的？

1 从"理解"到"生成"的问题

但还有一个关键问题没解决：这些组件是怎么组织起来的？

Transformer最初是为机器翻译设计的：输入一种语言的句子，输出另一种语言的句子。

这个任务需要两个阶段：

- 理解阶段：读懂输入的句子
- 生成阶段：写出输出的句子

对应到大脑，Transformer的发明者想出了"双脑"结构：



图1：Transformer的"双脑"结构——左边是Encoder（理解脑），右边是Decoder（生成脑）

1Encoder：理解的大脑

Encoder的任务是理解——把输入的文字变成计算机能理解的"知识"。

Encoder的结构：

- 输入嵌入 (Input Embedding) + 位置编码
- 多层 Self-Attention + 前馈网络
- 输出：每个词的"理解向量"

关键点：Encoder可以看到整个输入句子，所有的词可以互相"看到"彼此。

老师：Encoder做了什么？

小明：把句子读了一遍，理解了每个词的意思？

老师：准确地说，是把文字转换成了"知识表示"——一组向量。每个向量包含了这个词的语义信息，还有它和其他词的关系。

小明：所以Encoder的输出是"理解后的句子"？

老师：可以这么理解。就像你读完一篇文章，能回答关于这篇文章的问题——因为你已经在脑子里建立对这篇文章的理解了。

2 Decoder：生成的大脑

Decoder的任务是生成——根据已有的输入，预测下一个词该是什么。

Decoder的结构：

- 输出嵌入 (Output Embedding) + 位置编码
- 多层 Masked Self-Attention + Encoder-Decoder Attention + 前馈网络
- 输出：下一个词的概率分布

关键区别：Decoder是自回归生成——一次生成一个词，然后把生成的词作为下一轮的输入。

自回归生成 (Autoregressive Generation)

就像你写作文：一个字一个字地写，每写一个字，都要看前面所有的字来决定下一个字该写什么。

类比：就像天气预报员播报天气，每说一句话，都要结合之前说的内容，不能一次性把整篇稿子念完。

3 Encoder和Decoder是怎么配合的？

在机器翻译任务中，Encoder和Decoder的配合是这样的：

比如翻译："Hello, world!" → "你好，世界！"

4 为什么BERT只用Encoder，GPT只用Decoder？

这是现代大模型最重要的问题之一。

生活中的例子

想象你要写一篇作文：

- 如果老师让你"阅读一篇文章，然后回答问题"——你只需要理解，不需要创造。这就是BERT的模式。
- 如果老师让你"续写这个故事"——你需要理解前面的内容，然后创造新的内容。这就是GPT的模式。

BERT (Bidirectional Encoder Representations from Transformers) :

- 只用Encoder, 能看到整个输入
- 擅长: 文本分类、情感分析、问答、命名实体识别
- 特点: 需要labeled数据 (监督学习)

GPT (Generative Pre-trained Transformer) :

- 只用Decoder, 自回归生成
- 擅长: 文本生成、对话、写作、代码生成
- 特点: 可以从海量文本中学习 (自监督学习)

5不同的架构, 不同的应用

我的判断: 2022年之后, Decoder-only架构几乎一统天下。这是因为: (1) Scaling Law发现Decoder更容易scale up; (2) GPT-3的成功证明了纯生成路线可行; (3) Prompt Engineering让生成模型能做理解任务。但理解型任务用Encoder仍然更高效——不是所有问题都需要"生成"。

我的判断: 2022年之后, Decoder-only架构几乎一统天下。这是因为: (1) Scaling Law发现Decoder更容易scale up; (2) GPT-3的成功证明了纯生成路线可行; (3) Prompt Engineering让生成模型能做理解任务。但理解型任务用Encoder仍然更高效——不是所有问题都需要"生成"。

6为什么Decoder不能"作弊"?

Decoder在生成时, 必须一个一个词地预测, 不能"偷看"后面的内容。

这就是Masked Self-Attention的作用——它会遮住后面的词, 让Decoder只能看到前面的内容。

Masked Attention (带掩码的注意力)

Decoder生成"你好世界"时的注意力分布:

- 生成"你"时: 只能看到"你"
- 生成"好"时: 能看到"你"和"好"
- 生成"世"时: 能看到"你"、"好"、"世"
- 生成"界"时: 能看到"你"、"好"、"世"、"界"

类比: 就像盲人摸象, 每次只能根据已经摸到的部分来猜测下一部分是什么。这是"生成"的基本限制, 也是它

和"理解"的本质区别。

7总结

Encoder–Decoder的核心思想：Encoder负责理解，把文字变成"知识"；Decoder负责生成，把知识变成文字。两者配合，完成从输入到输出的转换。

Encoder–Decoder的核心思想：Encoder负责理解，把文字变成"知识"；Decoder负责生成，把知识变成文字。两者配合，完成从输入到输出的转换。

不同任务需要不同的架构：

- 理解型任务（分类、抽取）→ Encoder够用了
- 生成型任务（写作、对话）→ Decoder就够了
- 复转换任务（翻译、摘要）→ 两者都要

现代大模型大多走Decoder–only路线，因为Scaling

Law证明了这条路的有效性。但理解Encoder和Decoder的区别，能帮助你更好地理解不同模型的适用场景。

下篇预告

下期我们会讲一个新话题：Token——AI是怎么"认字"的。在理解了大模型的工作原理之后，我们需要搞清楚：大模型处理的到底是什么？答案就是Token。

系列五：大模型原理系列01 · Transformer架构原理02 · Self–Attention数学原理03 · 位置编码：让模型知道"谁在前面"04 · 多头注意力：让模型从多个角度理解05 · Encoder–Decoder架构：大模型的"双脑"结构 ← 你在这里06 · Token：AI是怎么"认字"的

6. 大模型原理系列 06

大模型原理系列 06

系列六Token —— AI是怎么"认字"的

我的观点：Token是大模型最基本的"原料"。很多人觉得大模型是直接处理文字的，其实不是——它处理的是Token。不理解Token，就像不理解"像素"就去做设计，早要踩坑。

我的观点：Token是大模型最基本的"原料"。很多人觉得大模型是直接处理文字的，其实不是——它处理的是Token。不理解Token，就像不理解"像素"就去做设计，早要踩坑。

先问大家一个问题

"你好"是多少个"字"？

你可能会说："两个字啊！"

但对AI来说，答案可能是——

不一定。AI用的单位叫"Token"，"你好"可能是2个Token，也可能是1个、3个，甚至更多。这取决于AI用什么方式把文字切成小块。

0上期回顾：Encoder和Decoder是怎么配合的

上期我们讲了Encoder-Decoder架构：Encoder负责"理解"，把文字变成知识；Decoder负责"生成"，把知识变成文字。

但不管是Encoder还是Decoder，它们处理的数据单位是什么？

答案是——Token。这是大模型最基本的"原料"。

1什么是Token？

Token是AI处理文本的基本单位。你可以把它理解为"AI认识的字"或者"文字的积木块"。

AI不是像人类一样一个字一个字地读，而是把文本切成一个个Token来处理。

生活中的类比

想象你教外国朋友学中文：

- 你说"苹果"——他们可能当成一个词来记
- 你说"水"——他们可能只记这一个字
- 你说"中华人民共和国"——他们可能记成5个词，也可能记成更少

同样的道理，AI把文字切成多少块，取决于怎么分最方便学习。

2英文怎么分Token？

英文的Tokenization通常基于"子词"（Subword）规则。

最常见的是Byte Pair Encoding（BPE）算法，简单说就是：

- 先统计哪些字符组合最常出现
- 然后把这些常用组合打包成一个Token
- 不常用的组合继续拆开

举个例子：

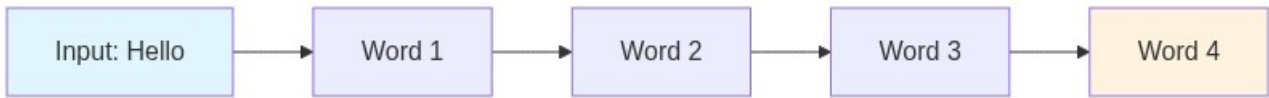


图1：BPE分词示例——常用词整个成Token，生僻词拆开成多个

3中文怎么分Token？

中文的分词更复杂，因为中文没有天然的空格分隔。

主流方法有几种：

- 基于词的分词：把句子切成"词"，比如"今天/天气/很好"
- 基于字的分词：每个汉字一个Token，比如"今/天/天/气/很/好"
- 混合分词：常用词作为一个Token，生僻字单独切

不同的模型可能用不同的策略：

老师：为什么"苹果"在中文里通常是一个Token？

小明：因为苹果是个常用词？

老师：没错！苹果是常用词，出现频率高，所以被当成一个整体来学习。如果是不常见的词，可能会被拆成"苹"和"果"两个Token。

4Token和字数的关系

这是大家最关心的问题：Token数和字数到底怎么换算？

对于中文，有一个大致的关系：

中文Token估算

- 1个汉字 $\approx$ 1–1.5个Token
- 1个英文单词 $\approx$ 1.3–1.5个Token
- 1个标点符号 $\approx$ 0.5–1个Token

粗略估算：1000个Token $\approx$ 500–750个中文字 $\approx$ 750个英文单词

但这只是估算！实际情况取决于：

- 文本内容（专业术语多的文本，Token可能更多）
- 分词算法（不同模型用的分词器不同）
- 语言混合（中英混的文本更难估算）

5Token为什么重要？

理解了Token，你会发现很多AI现象其实很正常：

现象1：为什么AI有时会"复读"？

比如你让它写诗，它可能重复"明月明月..."——这可能是因为某些Token组合特别常见，模型就学到了这种重复模式。

现象2：为什么中英文夹杂的文本处理效果差？

因为分词器通常针对一种语言优化，混合语言会增加分词难度。

现象3：为什么AI不认识你的名字？

生僻字或名字可能是分词器没学过的，就只能一个字一个字地处理，效果自然差。

Token的限制

每个AI模型都有"上下文窗口"限制，说白了就是一次能处理的Token数量有限：

- GPT-3.5：约4000个Token（一次大约3000字）
- GPT-4：约128000个Token（一次大约10万字）

这就是为什么超长文本需要分段处理——超出窗口的部分，AI"看不见"。

6Token的边界问题

Token切割有时候会产生有趣的问题：

问题1：文字游戏

"上海自来水来自海上"——正着读反着读意思不同。但对AI来说，如果"上海"是一个Token，它可能无法理解"海"在不同位置的含义差异。

问题2：算数难题

AI做数学题经常出错，因为数字的Token切分不符合数学直觉。比如"123"可能被切成"1"、"23"两个Token，而不是"一百二十三"这个整体概念。

为什么AI算数不行？

比如计算 $25 + 37 = ?$

人类可能直接算，但AI看到的是一串Token：

"25" 可能 = Token_A

"+" 可能 = Token_B

"37" 可能 = Token_C

它需要学会 Token_A + Token_B = Token_D

这种对应关系，但这种“学会”是靠统计规律，不是真正的数学能力。

类比：就像鹦鹉学舌，虽然能模仿说话，但不一定理解话的含义。

7Token Embedding：Token怎么变成数字？

分词只是第一步。切好的Token需要变成数字，AI才能处理。

这个过程叫Embedding——把每个Token映射到一个数字向量。

老师：Embedding是什么？

小明：就是把Token变成向量？

老师：对。每个Token对应一个向量，就像给每个词发一张"身份证"。相似的词，它们的向量也会相似。比如"猫"和"狗"的向量应该很近，因为都是动物。

小明：那这个词向量是怎么来的？

老师：训练出来的。模型在学习海量文本的过程中，自然而然地学会了：经常出现在相似上下文中的词，它们的向量也应该相似。这就是著名的"分布假说"——你应该从上下文理解一个词的意义。

8总结

Token的核心要点：1.

Token是AI处理文本的基本单位2.

中文通常是1个汉字=1个Token左右，但具体取决于分词器3。Token数 $\neq$ 字数，大致是1 Token $\approx$ 0.5-1.5个中文字4。每个AI模型有Token数量限制（上下文窗口）5。

每个AI模型有Token数量限制（上下文窗口）5.

Token的切分方式会影响AI对文本的理解

Token的核心要点：

1. Token是AI处理文本的基本单位
2. 中文通常是1个汉字=1个Token左右，但具体取决于分词器
3. Token数 $\neq$ 字数，大致是1 Token $\approx$ 0.5–1.5个中文字
4. 每个AI模型有Token数量限制（上下文窗口）

5. Token的切分方式会影响AI对文本的理解

理解Token，是理解大模型工作原理的关键一步。下次当你发现AI处理某些文本效果不好时，可以想想：是不是Token切分出了问题？

下篇预告

下期我们会讲：GPT是怎么生成文字的。了解了Token之后，我们就可以理解：AI是如何一个字一个字地"创作"出完整回答的？这个过程叫"自回归生成"。

系列六：大模型原理系列01 · Transformer架构原理02 · Self-Attention数学原理03 ·
位置编码：让模型知道"谁在前面"04 · 多头注意力：让模型从多个角度理解05 ·
Encoder-Decoder架构：大模型的"双脑"结构06 · Token：AI是怎么"认字"的 ← 你在这里07 ·
GPT生成原理：AI是怎么一个字一个字"写"出来的

7. 大模型原理系列 07

大模型原理系列 07

系列七GPT生成原理 —— AI是怎么一个字一个字"写"出来的

我的观点：很多人以为GPT是"一次性生成整段话"，其实不是。GPT是"一个字一个字"地生成——每次只预测下一个字是什么，然后把预测出来的字加到输入里，继续预测下一个。这个过程叫"自回归生成"，是理解ChatGPT工作原理的关键。

我的观点：很多人以为GPT是"一次性生成整段话"，其实不是。GPT是"一个字一个字"地生成——每次只预测下一个字是什么，然后把预测出来的字加到输入里，继续预测下一个。这个过程叫"自回归生成"，是理解ChatGPT工作原理的关键。

先问大家一个问题

你在ChatGPT里输入："请写一首关于春天的诗"

ChatGPT开始输出：

"春风吹绿了柳枝，桃花朵朵笑开颜..."

这些字是怎么出来的？

不是一次性想好整首诗，而是一个字一个字地"猜"出来的。每猜一个字，都要看前面已经写了什么，然后决定下一个字该写什么。

0上期回顾：Token是什么

上期我们讲了Token——AI处理文本的基本单位。不管是输入还是输出，AI都在处理Token。

但Token是怎么变成输出的？这就是今天要讲的：GPT的生成过程。

1自回归生成：一个字一个字地"猜"

GPT的核心工作方式是自回归生成（Autoregressive Generation）。

简单说就是：

- 输入一段文字
- 预测下一个字是什么
- 把预测的字加到输入后面
- 继续预测下一个字
- 重复这个过程，直到生成结束

生活中的类比

就像你写作文：

- 写完"今天"，想下一个字该写什么
- 写了"天气"，继续想下一个字
- 写了"很好"，再想下一个字...

你不是先把整篇作文想好再写，而是一边写一边想。GPT也是这样。

2具体过程演示

假设你输入："今天天气"

GPT的生成过程是这样的：

每一步，GPT都会计算所有可能的下一个字的概率，然后选择一个。

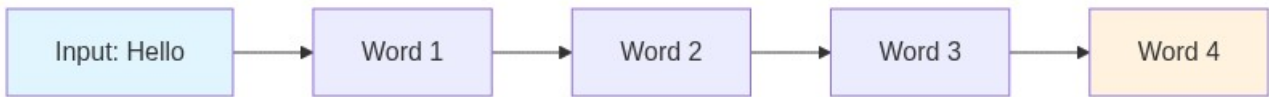


图1：自回归生成过程——每一步都基于前面已生成的内容预测下一个字

3为什么叫"自回归"？

"自回归"这个词听起来很高级，其实意思很简单：

自回归的含义

自：自己生成的输出，又成为下一轮的输入

回归：回到自己身上，用自己之前的结果来预测新的结果

类比：就像滚雪球——雪球越滚越大，因为它把自己滚过的雪都带上了。GPT也是，每生成一个字，这个字就加入"记忆"，用来生成下一个字。

老师：为什么GPT要用自回归方式生成？

小明：因为语言就是这样——每个字都和前面的字有关系？

老师：没错！语言是"序列数据"，有天然的顺序。"今天天气很好"和"很好天气今天"意思完全不同。自回归生成正是利用了这个顺序——每次只看前面的内容来预测下一个。

4概率分布：AI是怎么"选择"的？

每一步，GPT输出的不是一个字，而是一个概率分布——所有可能的下一个字，每个都有一个概率值。

比如输入"今天天气"，GPT可能输出：

- "很"：30%
- "不错"：20%
- "真好"：15%
- "晴朗"：10%
- ...（其他几万个字，每个都有概率）

然后，GPT会根据这个概率分布采样一个字。采样方式有几种：

5温度参数：控制AI的"创意度"

你可能见过ChatGPT有个"temperature"参数。这是干什么的？

温度（Temperature）控制概率分布的形状：

- 温度 = 0：永选概率最高的字（最保守，最确定）
- 温度 = 0.7：稍微有点随机（平衡，ChatGPT默认值）
- 温度 = 1.0：完全按原始概率（更有创意，但也更可能出错）
- 温度 > 1：非常随机（可能胡言乱语）

温度的数学原理

原始概率分布经过温度调整后：

新的概率 = $\exp(\text{原始分数} / \text{温度}) / \text{归一化常数}$

类比：温度就像"放大镜"。温度低，放大镜把高概率的字放得更大，低概率的字几乎看不见；温度高，所有字的概率都变得差不多，选择就更随机。

6生成结束：什么时候停止？

GPT不会一直生成下去，它需要知道什么时候该停。

停止条件有几种：

- 遇到结束符：GPT训练时学过一个特殊的"结束Token"，生成到这个Token就停
- 达到最大长度：比如限制最多生成1000个字
- 用户中断：用户点击"停止生成"按钮

7为什么GPT能"续写"？

理解了自回归生成，你就明白了为什么GPT擅长"续写"任务。

因为续写本质上就是：给定前面的内容，预测后面的内容——这正是自回归生成在做的事！

不管是：

- 写代码（给定函数名，续写函数体）
- 写文章（给定标题，续写正文）
- 对话（给定问题，续写回答）

都是同一个原理：把已有内容当输入，预测下一个字，循环往复。

8总结

GPT生成的核心原理：1. 自回归生成：一个字一个字地预测2. 每次预测输出概率分布，然后采样选择3. 生成的字加入输入，继续预测下一个4. 温度参数控制"创意度"（保守 vs 随机）5. 遇到结束符或达到长度限制时停止

GPT生成的核心原理：

1. 自回归生成：一个字一个字地预测
2. 每次预测输出概率分布，然后采样选择

3. 生成的字加入输入，继续预测下一个
4. 温度参数控制"创意度"（保守 vs 随机）
5. 遇到结束符或达到长度限制时停止

理解了自回归生成，你就理解了ChatGPT是怎么"说话"的——它不是一次性想好整段话，而是一边说一边想，每说一个字都要看前面说了什么。

下篇预告

下期我们会讲：为什么模型越大越聪明——Scaling

Law（缩放定律）。为什么GPT-3有1750亿参数？为什么参数越多效果越好？这背后有什么规律？

系列七：大模型原理系列01 · Transformer架构原理02 · Self-Attention数学原理03 ·
位置编码：让模型知道"谁在前面"04 · 多头注意力：让模型从多个角度理解05 ·
Encoder-Decoder架构：大模型的"双脑"结构06 · Token：AI是怎么"认字"的07 ·
GPT生成原理：AI是怎么一个字一个字"写"出来的 ← 你在这里08 · Scaling Law：为什么模型越大越聪明

8. 大模型原理系列 08

大模型原理系列 08

系列八Scaling Law —— 为什么模型越大越聪明

我的观点：Scaling

Law（缩放定律）是大模型时代最重要的发现之一。它告诉我们：模型效果会随着参数量、数据量、计算量的增加而可预测地提升。这不是玄学，而是有数学规律可循的。理解了Scaling

Law，你就理解了为什么OpenAI要不断把模型做大。

我的观点：Scaling

Law（缩放定律）是大模型时代最重要的发现之一。它告诉我们：模型效果会随着参数量、数据量、计算量的增加而可预测地提升。这不是玄学，而是有数学规律可循的。理解了Scaling

Law，你就理解了为什么OpenAI要不断把模型做大。

先问大家一个问题

GPT-3有1750亿参数，GPT-4据说更多。

为什么要把模型做得这么大？

你可能会说："参数多，能力就强呗。"

但为什么参数多就能力强？有没有上限？——

答案在Scaling

Law里。OpenAI在2020年发表的研究发现：模型效果和参数量、数据量、计算量之间存在幂律关系——可以预测！

0上期回顾：GPT是怎么生成的

上期我们讲了GPT的自回归生成：一个字一个字地预测，每次都基于前面的内容。

但还有一个问题：为什么GPT-3能生成得这么好？

答案：因为它够大。但"大"和"好"之间是什么关系？这就是Scaling Law要回答的问题。

1什么是Scaling Law？

Scaling Law是OpenAI在2020年发现的经验规律：

Scaling Law的核心公式

损失函数 $L(N, D, C) \approx L_{\infty} + (N/N^*)^{(-\alpha_N)} + (D/D^*)^{(-\alpha_D)} + (C/C^*)^{(-\alpha_C)}$

其中：

- N = 参数量 (Parameters)
- D = 数据量 (Data size)
- C = 计算量 (Compute)

简单说：模型效果（损失函数）会随着 N 、 D 、 C 的增加而下降，而且这个下降是可预测的——不是随机变好，而是有规律地变好。

生活中的类比

想象你在考试：

- 学习时间越长，成绩越好
- 练习题越多，成绩越好
- 脑子越聪明，成绩越好

Scaling Law就是发现了这三者和成绩之间的数学关系——不是"可能变好"，而是"一定会变好多少"。

2三个关键因素

Scaling Law告诉我们，模型效果取决于三个因素：

老师：这三个因素哪个最重要？

小明：参数量？因为GPT-3就是因为参数多才厉害？

老师：其实三者要平衡。Scaling

Law发现：给定计算预算，最优的参数量和数据量有一个固定比例。只增加参数不增加数据，效果提升会变慢

。

3幂律关系：效果提升有规律

Scaling Law最惊人的发现是：效果提升遵循幂律（Power Law）。

这意味着：

- 参数量增加10倍 → 损失下降固定比例
- 数据量增加10倍 → 损失下降固定比例
- 计算量增加10倍 → 损失下降固定比例

而且这个比例是可预测的！

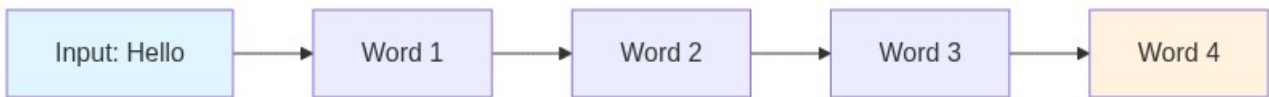


图1：Scaling Law曲线——损失随参数量/数据量/计算量增加而下降，呈幂律关系

4为什么GPT-3有1750亿参数？

理解了Scaling Law，你就明白了为什么GPT-3要这么大。

OpenAI的计算：

- 根据Scaling Law，要达到某个效果水平，需要多少参数？
- 给定计算预算（训练成本），最优的参数量是多少？
- 1750亿参数，是在当时计算预算下的最优选择

这不是拍脑袋决定的，而是根据Scaling Law计算出来的。

模型规模的演进

- GPT-1：1.17亿参数（2018）
- GPT-2：15亿参数（2019）
- GPT-3：1750亿参数（2020）
- GPT-4：未公开，估计万亿级别（2023）

规律：每个版本大约增加10倍参数。这不是巧合，而是Scaling Law告诉他们：要提升效果，就得这么增加。

5涌现现象：量变到质变

Scaling Law预测的是平均效果，但还有一个有趣的现象：涌现（Emergence）。

某些能力不是线性提升的，而是在达到某个规模后突然出现：

涌现的例子

研究发现：

- 3B参数以下的模型：几乎不会做三位数加法
- 10B参数：突然能做三位数加法了
- 100B参数：能做更复杂的数学推理

这种"突然出现"的能力，就是涌现现象——量变到质变。

6Scaling Law的局限

Scaling Law不是万能的：

- 只适用于训练损失：预测的是训练集上的效果，实际任务效果可能有差异
- 不看架构改进：比如Transformer比RNN好，这不是Scaling Law能预测的
- 计算成本巨大：按Scaling Law继续扩大，成本可能无法承受
- 数据瓶颈：高质量数据可能不够用了

我的判断：Scaling

Law告诉我们"做大有用"，但没有告诉我们"做大的上限在哪"。2023年之后，很多研究者开始探索：能不能用更少的参数达到同样的效果？这就是"高效训练"、"模型压缩"等方向的研究动机。

我的判断：Scaling

Law告诉我们"做大有用"，但没有告诉我们"做大的上限在哪"。2023年之后，很多研究者开始探索：能不能用更少的参数达到同样的效果？这就是"高效训练"、"模型压缩"等方向的研究动机。

7总结

Scaling Law的核心要点：1. 模型效果与参数量、数据量、计算量呈幂律关系2. 效果提升是可预测的，不是随机变好3. 三者要平衡，不能只增加一个4. 某些能力会在达到某个规模后"涌现"5. 但Scaling Law有局限，不能无限扩大

Scaling Law的核心要点：

- 1. 模型效果与参数量、数据量、计算量呈幂律关系
- 2. 效果提升是可预测的，不是随机变好
- 3. 三者要平衡，不能只增加一个
- 4. 某些能力会在达到某个规模后"涌现"
- 5. 但Scaling Law有局限，不能无限扩大

理解了Scaling

Law，你就理解了大模型发展的底层逻辑——为什么大家都在"卷参数"、"卷数据"、"卷算力"。因为Scaling Law告诉他们：这样做，效果一定会提升。

系列八：大模型原理系列01 · Transformer架构原理02 · Self-Attention数学原理03 · 位置编码：让模型知道"谁在前面"04 · 多头注意力：让模型从多个角度理解05 · Encoder-Decoder架构：大模型的"双脑"结构06 · Token：AI是怎么"认字"的07 · GPT生成原理：AI是怎么一个字一个字"写"出来的08 · Scaling Law：为什么模型越大越聪明 ← 你在这里

9. 大模型原理系列 · 开篇

大模型原理系列 · 开篇

从"会用"到"真懂"——Agent开发者的最后一公里

1一个老面试题的启发

"你了解GPT是怎么生成文字的吗？"

这个问题，我问过很多AI应用开发者。大多数人的回答是："就是下一个字下一个字生成的。"

再追问几句，很多人就答不上来了。

这很正常。毕竟会用和真懂之间，隔着一整个技术原理的世界。

我们用了三个系列的时间，从用户走到创造者：

- 系列一，我们学会了用OpenClaw，从部署到自定义开发
- 系列二，我们学会了用AI Coding，做完整的项目级开发
- 系列三，我们学会了造Agent，理解了执行引擎、记忆系统、工具调用、多Agent协作

但你有没有想过一个问题：为什么你的Agent有时候会'智障'？

为什么Prompt写得好与不好，效果天差地别？为什么上下文窗口那么大，Agent还是容易遗忘？为什么RAG听起来简单，真正用起来却效果不定？为什么同样的模型，别人的Agent就是比你的聪明？

为什么Prompt写得好与不好，效果天差地别？为什么上下文窗口那么大，Agent还是容易遗忘？为什么RAG听起来简单，真正用起来却效果不定？为什么同样的模型，别人的Agent就是比你的聪明？

这些问题，不在Agent层解决，而在更底层——在大模型层。

2三个系列学的是什么

关键词：平台使用、Bot设计、插件开发

你会用OpenClaw了，知道怎么创建Bot、配置插件、发布 workflow

关键词：项目开发、前端工程、UI组件

你会用AI工具做完整项目了，知道怎么设计架构、编写代码、调试bug

关键词：执行引擎、记忆系统、工具调用、多Agent协作

你会造Agent应用了，知道怎么让Agent思考、记忆、调用工具、协同工作

但这三个系列，都建立在一个前提上：有一个能力足够强的大模型在背后支撑。

我们一直在使用大模型，却从来没有理解大模型。

3为什么要理解大模型

让我给你几个具体的场景：

你在系列三里实现了上下文窗口管理（第9篇），学会了控制Token数量来优化成本。但你有没有想过：上下文窗口到底是怎么工作的？为什么模型能"看到"很长的对话，却对处的信息记忆模糊？

你在系列三里实现了上下文窗口管理（第9篇），学会了控制Token数量来优化成本。但你有没有想过：上下文窗口到底是怎么工作的？为什么模型能"看到"很长的对话，却对处的信息记忆模糊？

这需要理解Attention机制——Transformer的核心。

你在系列三里实现了RAG知识库（第11篇），学会了让Agent从文档中检索答案。但你有没有想过：Embedding是怎么把文字变成向量的？相似度搜索为什么能找到相关的内容？

你在系列三里实现了RAG知识库（第11篇），学会了让Agent从文档中检索答案。但你有没有想过：Embedding是怎么把文字变成向量的？相似度搜索为什么能找到相关的内容？

这需要理解Embedding和向量检索——语义搜索的基础。

你在系列三里实现了Agent自我进化（第20篇），想让Agent能不断学习。但你有没有想过：模型是怎么从人类的反馈中学习的？为什么有些模型特别"听话"，有些却很难调教？

你在系列三里实现了Agent自我进化（第20篇），想让Agent能不断学习。但你有没有想过：模型是怎么从人类的反馈中学习的？为什么有些模型特别"听话"，有些却很难调教？

这需要理解RLHF和对齐技术——让模型更懂人类意图。

4这个系列要讲什么

大模型原理系列，共12篇文章，承上启下，每篇独立发布。

5学完这个系列你能做什么

不再是"调API的"，而是真正懂AI应用内核的开发者。

- 更精准的Prompt：理解Attention原理后，你知道怎么设计让模型"重点关注"关键信息
- 更稳定的RAG：理解Embedding原理后，你知道怎么优化切分策略和检索逻辑
- 更聪明的Agent：理解模型能力边界后，你知道什么该让Agent做，什么该让模型做
- 更高效工程：理解Tokenizer后，你能精准控制成本；理解推理优化后，你能提升响应速度
- 更安全的应用：理解对齐技术后，你知道怎么防止Prompt注入和模型越狱

更重要的是：当你面试时，能真正回答出"GPT是怎么生成文字的"——不是死记硬背，而是真正理解。

更重要的是：当你面试时，能真正回答出"GPT是怎么生成文字的"——不是死记硬背，而是真正理解。

6写在最后

四个系列，终于要闭环了：

系列一看清平台→ 系列二掌握工具→ 系列三学会造应用→ 系列四理解大脑

学完这四系列，你不再是"调API的程序员"，而是真正能从底层理解AI应用、从顶层设计系统的AI应用开发专家。

这不是一条轻松的路。但既然你已经走完了前三个系列，这条路对你来说，不会有问题。

下期预告：第1篇 · Transformer架构原理——从 RNN 到 Attention，模型是怎么"学会"理解语言的。

大模型原理系列 · 开篇从"会用"到"真懂"，我们一起走完这最后一公里

10. 大模型原理系列 · 总结

大模型原理系列 · 总结

系列总结从Transformer到GPT，8篇文章读懂大模型原理

写在前面：这个系列从Transformer架构讲起，一路讲到Scaling

Law。8篇文章，覆盖了大模型的核心原理。如果你从头看到尾，现在应该能回答这个问题了："ChatGPT是怎么工作的？"

写在前面：这个系列从Transformer架构讲起，一路讲到Scaling

Law。8篇文章，覆盖了大模型的核心原理。如果你从头看到尾，现在应该能回答这个问题了："ChatGPT是怎么工作的？"

1知识脉络回顾

整个系列的知识结构是这样的：

第一层：基础架构

01 Transformer架构 → 为什么不用RNN？并行计算的优势

第二层：核心组件

03 位置编码 → 解决"谁在前面"的问题

04 多头注意力 → 让AI有"多个脑子"同时分析

第三层：架构变体

05 Encoder-Decoder → 理解脑 vs 生成脑（BERT vs GPT）

第四层：数据与生成

06 Token → AI处理文本的基本单位

07 GPT生成原理 → 自回归生成，一个字一个字预测

第五层：规模效应

08 Scaling Law → 为什么模型越大越聪明

2核心概念速查表

3现在你能回答这些问题了

Q1：ChatGPT是怎么生成回答的？

自回归生成——一个字一个字预测，每次都基于前面已经生成的内容。每预测一次，输出所有可能字的概率分布，然后采样选择一个。

Q2：为什么GPT-3有1750亿参数？

Scaling Law告诉OpenAI：要达到某个效果水平，需要多少参数。1750亿是在当时计算预算下的最优选择。

Q3：BERT和GPT有什么区别？

BERT只用Encoder，擅长理解（分类、问答）；GPT只用Decoder，擅长生成（写作、对话）。

Q4：为什么AI有时会"复读"？

可能是某些Token组合特别常见，模型学到了这种重复模式。调整温度参数可以缓解。

Q5：为什么AI算数不行？

数字的Token切分不符合数学直觉。比如"123"可能被切成"1"和"23"两个Token，AI需要学的是统计规律，不是真正的数学。

4学习建议

给不同读者的建议

如果你想深入理解：

- 补量自己实现一个迷你Transformer（50行代码）
- 读原始论文《Attention Is All You Need》
- 看OpenAI的Scaling Law论文

如果你只想应用：

- 记住：Token、自回归生成、温度参数
- 理解：为什么大模型要做得这么大
- 知道：不同任务该用什么模型（BERT vs GPT）

如果你想继续学习：

- 研究方向：高效训练、模型压缩、多模态
- 实践方向：微调、Prompt Engineering、RAG

5系列完整目录

基础架构层01 · Transformer架构原理02 · Self-Attention数学原理（待补充）核心组件层03 ·
位置编码：让模型知道"谁在前面"04 · 多头注意力：让模型从多个角度理解架构变体层05 ·
Encoder-Decoder架构：大模型的"双脑"结构数据与生成层06 · Token：AI是怎么"认字"的07 ·
GPT生成原理：AI是怎么一个字一个字"写"出来的进阶原理层08 · Scaling Law：为什么模型越大越聪明

最后的话：大模型不是魔法，是工程。理解了原理，你就能更好地使用它、优化它、甚至改进它。希望这个系列对你有帮助！

最后的话：大模型不是魔法，是工程。理解了原理，你就能更好地使用它、优化它、甚至改进它。希望这个系列对你有帮助！

大模型原理系列 · 完结感谢阅读